Mimicry — Demo 2

What	Where
Demo 1 — record → form → REST	https://github.com/adittosarkerr/mimic
Demo 2 — voice enabled	https://github.com/adittosarkerr/mimicry
Live deployment	https://mimicry-runner-jwh3.vercel.app
Video walkthrough	*(see §3 — to be recorded)*

1. How this version integrates voice

Demo 1 had one way in: record a task with the browser extension, and the recording becomes a form plus a REST endpoint. That still works unchanged. Voice is a **second front door onto the same machinery**, not a layer bolted over it.

The path a sentence takes

```
speech ──▶ transcript ──▶ plan ──▶ automation ──▶ run ──▶ results
              repair        match or author          the same engine
                                                     Demo 1 uses
```

What voice added back to the recorded path

Voice work turned up faults that were in Demo 1 all along, and the fixes went into the shared code, so recorded automations get them too:

- **Counters are one control.** "3 adults" was four anonymous clicks on a `+` — unreplayable, and not a field. It is now a single step that compiles to a number.
- **Sites whose URLs weld values together** (GoZayaan `trips=DAC,BKK,2026-10-05`, Kayak, Booking) get hand-written profiles. No `{placeholder}` reaches inside those, so inference produced forms that searched the recorded trip forever whatever was typed.
- **Filters are read from the request.** Booking has 14 filters wired, every code read off the live page and confirmed by applying it and watching the result count move. Anything unrecognised is shown and labelled *not applied*, rather than dropped silently.
- **A site that refuses to render** is read another way. Daraz serves a headless browser a complete page with no product grid and no error; the structural scanner found its top bar and reported "8 products" with a QR code. It is now read from the JSON the page fetches for itself.

2. Tests run to confirm Demo 1 still works

Everything below was executed against live sites, not mocked. Numbers are from actual runs.

2.1 Extraction accuracy — the core Demo 1 promise

`node scripts/extract-suite.mjs` drives the real extractor across 14 sites.

Result	Count
Good	11
Weak	1 (FitGirl — 3 real results, genuinely few)
Refused by the site	2 (eBay, Reddit — reported honestly, not as "no results")

Sample: YouTube 17 videos · Wikipedia 20 · Booking 26 stays · Amazon 15 products · Google News 100 articles · Hacker News 30 · StackOverflow 15 · GitHub 10 repos.

This suite caught three real regressions during the work, all fixed:

- Amazon returned nothing because the page says *"1-16 of over 40,000 results"*, which contains `0 results` — the empty-state hint. Fifteen products discarded on the last digit of forty thousand.
- Wikipedia, DuckDuckGo and FitGirl returned nothing because `closest()` reaches the root element and Wikipedia's skin writes `vector-feature-main-menu-pinned` onto `<html>` — so every element on the page counted as inside a "menu".
- A page of search results was being read as an article (866 words of site chrome, twenty real results ignored).

2.2 Recorded replay, end to end

Automation	Result
GoZayaan round trip DAC→NYC	28 distinct itineraries, all priced
GoZayaan one-way DAC→BKK	59 flights, single leg, correct route
Booking Cox's Bazar, 5★ + breakfast	3 hotels with prices and ratings, 12s
Booking New Delhi + swimming pool	`hotelfacility=433` present only when asked
Daraz "passport holder", cheapest first	400 products, 10 pages, 400/400 images, strictly ascending

2.3 Shared rules — accounts, marketplace, billing, quota

`npx tsx scripts/core-rules-test.mts` — **28 checks, all passing**, including the paths that only exist to be tested: a card ending `0000` declines, the failed charge still writes a receipt, cancelling someone else's subscription is refused, and the sixth run of a five-run day is blocked.

2.4 Site profiles

`npx tsx scripts/profile-mapping-test.mts` — **9/9 Booking fields** map correctly from the names a model actually produces (`star_rating` → `stars` , `breakfast_included` → `breakfast`).

`npx tsx scripts/daraz-test.mts` — 120 items across 3 pages, all priced, all with absolute URLs, no navigation leaking in, and a nonsense query returning zero with a real explanation.

2.5 Filter codes verified against the live site

`npx tsx scripts/verify-filters.mts` applies one filter at a time and records how many of ~83 Cox's Bazar properties survive. An invented token does not error on Booking — it silently returns everything — so a number that moved is the only proof worth having.

Filter	Code	Matched
Swimming pool	`hotelfacility=433`	13
Beachfront	`hotelfacility=146`	9
Spa	`hotelfacility=54`	10
Free WiFi	`hotelfacility=107`	61
Parking	`hotelfacility=2`	77
Air conditioning	`roomfacility=11`	73
Breakfast	`mealplan=1`	49
Free cancellation	`fc=2`	41
No prepayment	`oos=1`	83
5 stars	`class=5`	4

2.6 Result ordering

Compared position-for-position against Daraz's own JSON: **identical, 40/40**. Explicit sorts honoured in both directions — `priceasc` first prices 19, 20, 25.5; `pricedesc` first prices 6530, 5838, 3148.

2.7 Fresh-clone install

Cloned the public repo to a clean directory, installed, and ran without touching the working copy:

- Runner starts with **no configuration at all** (reports `AI compiler → disabled`)
- With `.env.local`: authored a Booking automation from a sentence in **4.5s** on an empty store
- Ran it: **3 hotels, 14.6s**
- `next build` clean, all 16 routes

2.8 Deployment

Verified live on Vercel with no runner: `store: supabase`, `browser: true`, voice authoring, a YouTube run returning **92 results in 54s**, recording ingest, marketplace and the payment sandbox.

2.9 Type and build integrity

`tsc --noEmit` clean on both `apps/runner` and `apps/web`; `next build` clean with no warnings, after every change above.

---

3. Video walkthrough

> **This section needs your recording — I cannot produce a video.**
>
> Replace the link below once uploaded (YouTube unlisted or Drive both work).

Suggested running order, matching what was asked for:

1. **API creation, Demo 1** — record a task with the extension, show the generated form and the `POST /api/automations/:id/run` endpoint.
2. **API creation, Demo 2** — say the same task out loud on `/voice`, show the plan card, run it, show the identical REST endpoint.
3. **Both APIs, three searches:**

Query	What to point out
`passport holder`	400 products, 10 pages, prices and ratings, the pager
`dsfkjlskdfj;lkdsjfslkdjfslkdfj;lsdlkjfs;ldkfjs;ldkjfs;ldkjfskdlfjs;dlfjs;ldkjf;ldslkjfd`	Returns **zero** with *"Daraz returned no products for this search. The site itself reports 0 results."* — a real empty state, not the site's furniture
`MacBook Pro`	Real products with prices; compare Demo 1's recorded form against Demo 2's spoken request

The nonsense query is the one worth dwelling on. Before this work, a search Daraz could not answer came back as eight "products" that were the site's own top bar — "SAVE MORE ON APP", "BECOME A SELLER", a QR code — reported as success. A confident wrong answer is worse than any error, because every other failure tells you something went wrong.

Known limits

Stated plainly, because each is a real boundary rather than a bug awaiting a fix.

- **Serverless runs have ~45 seconds.** Vercel stops a function at 60 on the Hobby plan. Most runs fit; a 400-product catalogue does not. The run stops at its budget and says so rather than being killed. The container runner has no limit.
- **Booking refuses datacenter IPs.** It renders 26 properties locally in 10 seconds and none from Vercel in 18. The deployed site says so instead of inventing results.
- **The live console needs a socket**, which a serverless function has not. Results arrive complete at the end there.
- **GoZayaan has no URL filters.** Its only narrowing controls are departure-time and duration buckets, applied in-page with generated ids. Adding fields for them would recreate the exact problem of controls that quietly do nothing.
- **Multi-step builders** (a PC configurator) are beyond a single search-and-scrape automation.